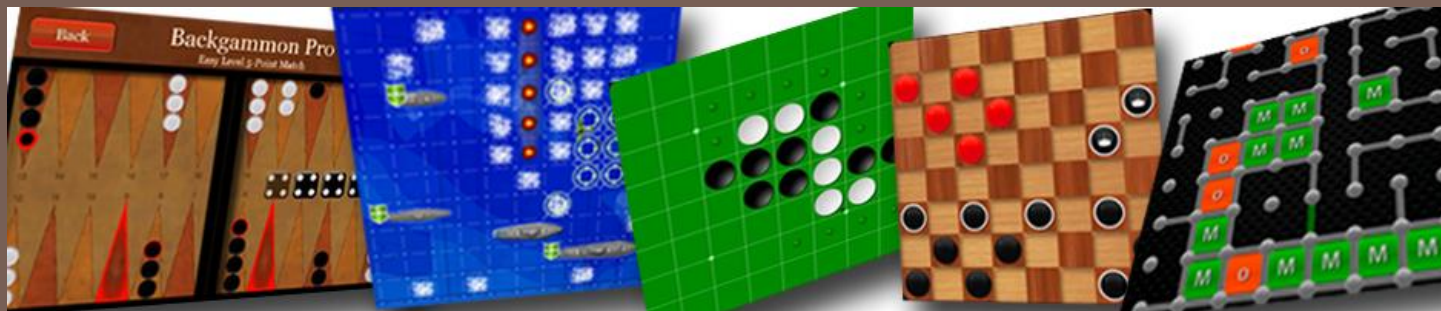


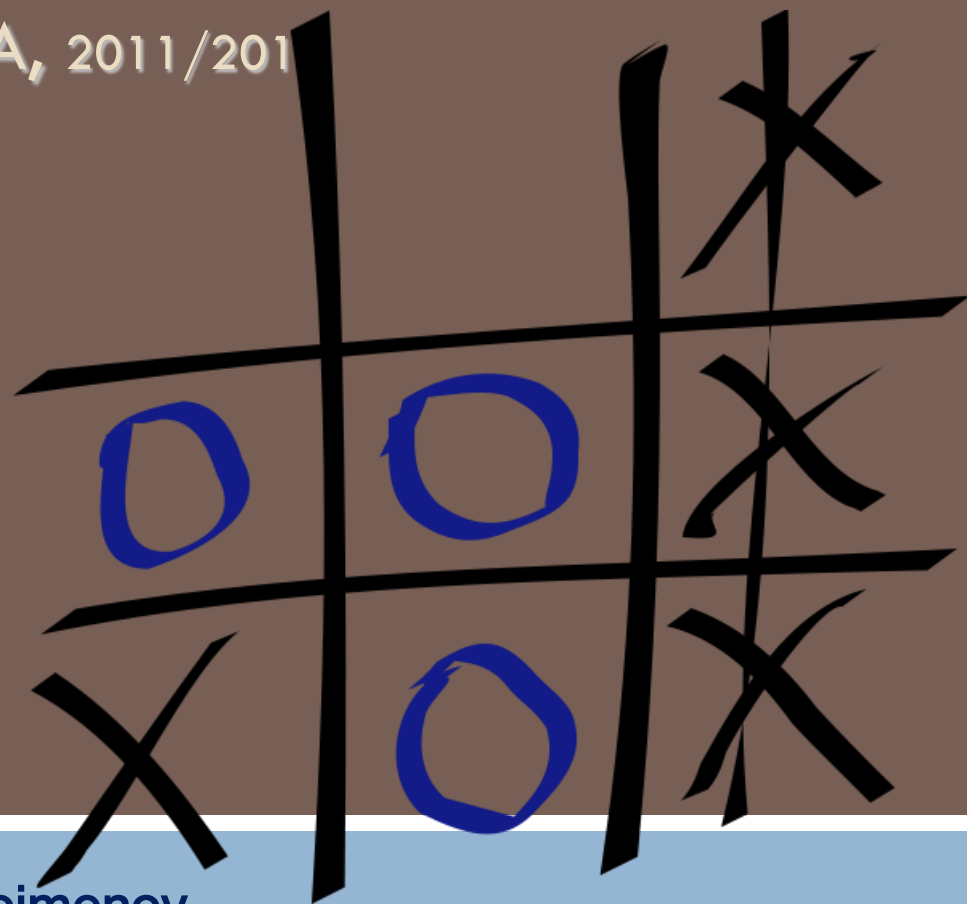


# VEŠTAČKA INTELIGENCIJA, 2011/2012

## TRAŽENJE I IGRE

### Sadržaj

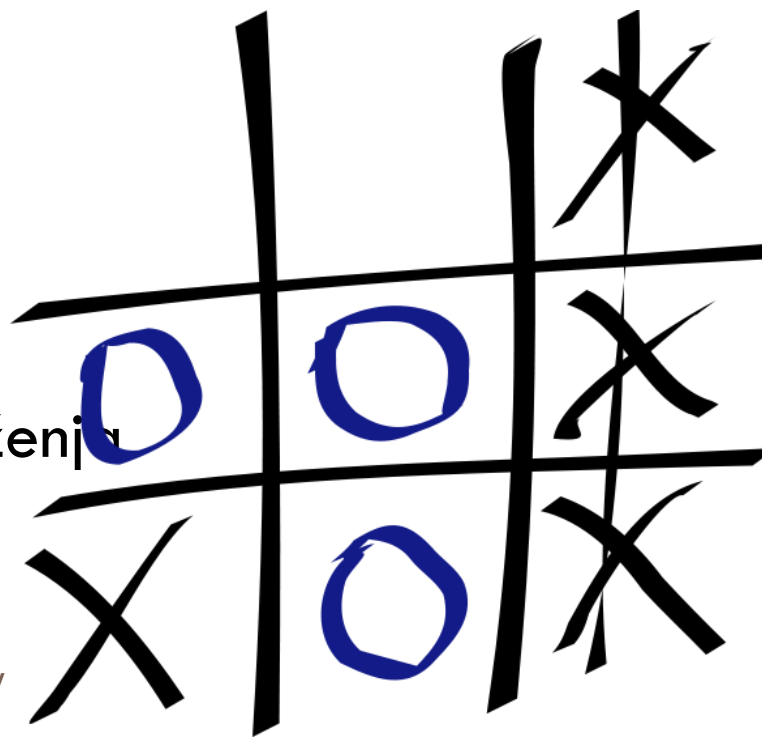
- MinMax algoritam
- Alfa-Beta odsecanje



Prof. dr Leonid Stoimenov

# Motivacija

- Kreiranje programa koji igraju igre (logičkog tipa)
- Treba da igra optimalno
- Treba da daje odgovore u razumnom vremenskom periodu
- Razmatramo:
  - ▣ MinMax algoritam
  - ▣ MinMax sa ograničavanjem traženja
    - Alfa-Beta odsecanje



# Tipovi igara

|                         | Deterministic                      | Nondeterministic<br>(Chance) |
|-------------------------|------------------------------------|------------------------------|
| Fully<br>Observable     | Chess<br>Checkers<br>Go<br>Othello | Backgammon<br>Monopoly       |
| Partially<br>Observable | Battleship                         | Card Games                   |

**Minimax** je za igre bez slučajnih elemenata,  
samo jedan je pobednik

# Traženje i Algoritmi za Igre

Šah, 8-puzzle, tic-tac-toe, mankala,...

- Dva igrača (MIN I MAX) naizmenično povlace poteze
  - ▣ Max uvek igra prvi.
  - ▣ Min je protivnik.

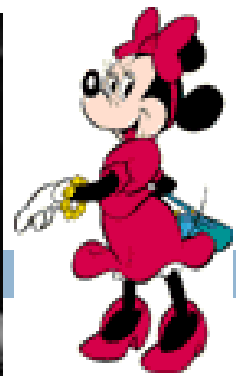
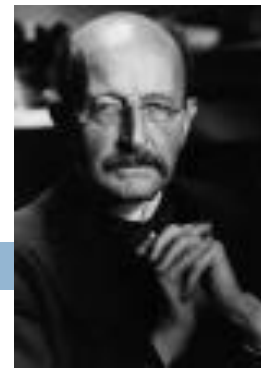
## Definisanje preko traženja:

- Početno stanje
- Skup operatora
- Test ciljnog stanja (pobeda ili poraz)
- Funkcija korisnosti tj evaluacije (utility function)



Max Vs Min

# Funkcije korisnosti



Max Vs Min

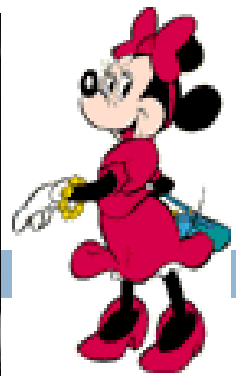
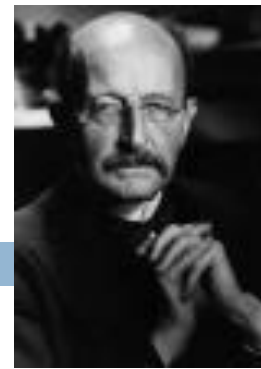
**Funkcija korisnosti** je kao heuristička funkcija s tom razlikom što se preko nje vrši evaluacija čvora (koji se odnosi na potez nekog igrača) u smislu **procene** koliko je on **koristan za svakog od igrača**

**Pozitivne** vrednosti indiciraju stanja koja obezbeđuju **prednost za Max**

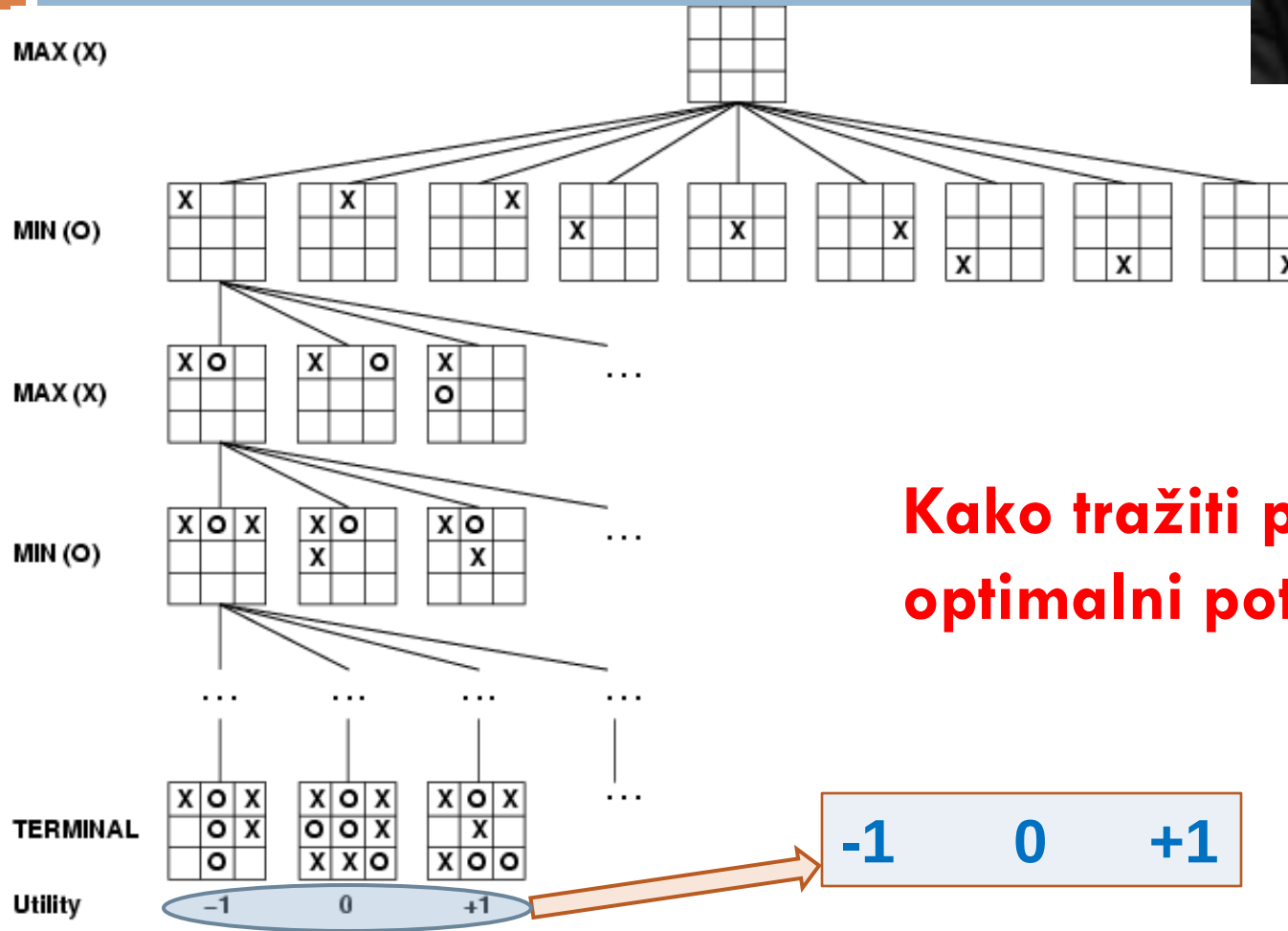
**Negativne** vrednosti indiciraju stanja koja obezbeđuju **prednost za Min**

+1, -1, 0 za mnoge igre

# Stablo traženja za igru (2-igrača, deterministička)



Max Vs Min



**Kako tražiti po stablu za optimalni potez?**

# Minimax algoritam – osnovna ideja

- Generiše kompletno stablo traženja
  - ▣ Na svakom nivou jedan od igrača povlači potez
  - ▣ Traženje po stablu mogućih poteza sa ciljem da se nađe potez koji prouzrokuje najbolji rezultat
- Analiza celokupnog stabla daje optimalne poteze (za oba igrača – pretpostavka da i protivnik igra optimalno!)
- Depth First Search (DFS) algoritam
- Generalno nije izvodljivo – stablo može biti preveliko

# Postavke za Minimax

- Inicijalno stanje
  - Pozicija tj stanje na tabli
  - Ko je na potezu
- Operatori
  - Legalni potezi igrača
- Test
  - Određuje da li je stanje ciljno
- Funkcija korisnosti (zavisi od igre)

# Funkcija korisnosti (evaluacije)

- Procenjuju vrednost stanja na osnovu njegovih osobina – daje korisnost nekog stanja igre (**utility(State)**)
- Primer: -1, 0, +1: Igrač 1 gubi, nerešeno, Igrač 1 dobija (respektivno)
- Linearna težinska funkcija (napr. Šah)

$$\text{utility}(S) = w_1 f_1(S) + w_2 f_2(S) + \dots + w_n f_n(S)$$

- $f_1(S) = (\text{Number of white queens}) - (\text{Number of black queens}),$   
 $w_1 = 9$

- $f_2(S) = (\text{Number of white rooks}) - (\text{Number of black rooks}),$   
 $w_2 = 5$

- ...

# Implementacija: Dva agenta

## □ MAX

- ▣ Pokušava da maksimizuje rezultat funkcije korisnosti
- ▣ Pobednička strategija, ako, na MIN potez po redu, pobjeda je dostižna za MAX za sve moguće poteze MIN

## □ MIN

- ▣ Pokušava da minimizuje rezultat funkcije korisnosti
- ▣ Pobednička strategija, ako, na MAX potez po redu, pobjeda je dostižna za MIN, za sve moguće poteze MAX

# MinMax algoritam

```
function MINIMAX-DECISION(state) returns an action  
  inputs: state, current state in game  
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))
```

---

```
function MAX-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$   
  return v
```

---

```
function MIN-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow \infty$   
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$   
  return v
```

# MinMax algoritam

Maks i Mini igraju igru

**P (list\_stabla) =**

- 1 ako Maks pobedi
- 0 za nerešeno
- 1 ako Maks gubi, tj Mini dobija

**Čvor V(X):**

$V(X) =$

ako je X list stabla

$P(x)$

inače

$\max \{V(c) \mid c \text{ je potomak } X\}$   
ako Maks igra sledeći

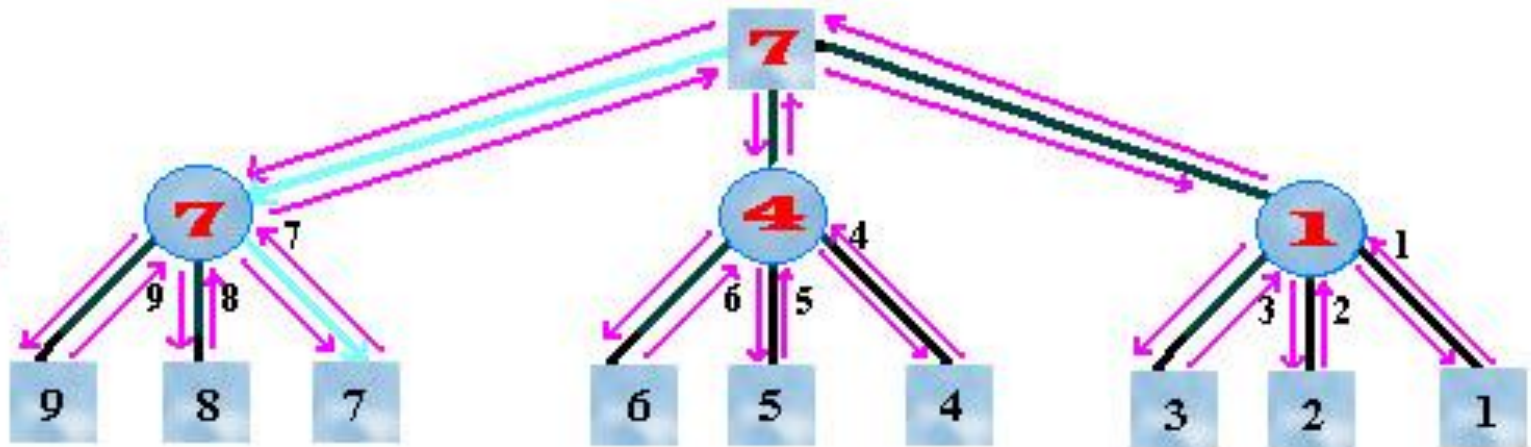
$\min \{V(c) \mid c \text{ je potomak } X\}$   
ako Mini igra sledeća

# MinMax traženje

MAX  
You::

MIN  
Opponent::

MAX  
You::



MIN



MAX



PATH (stored)

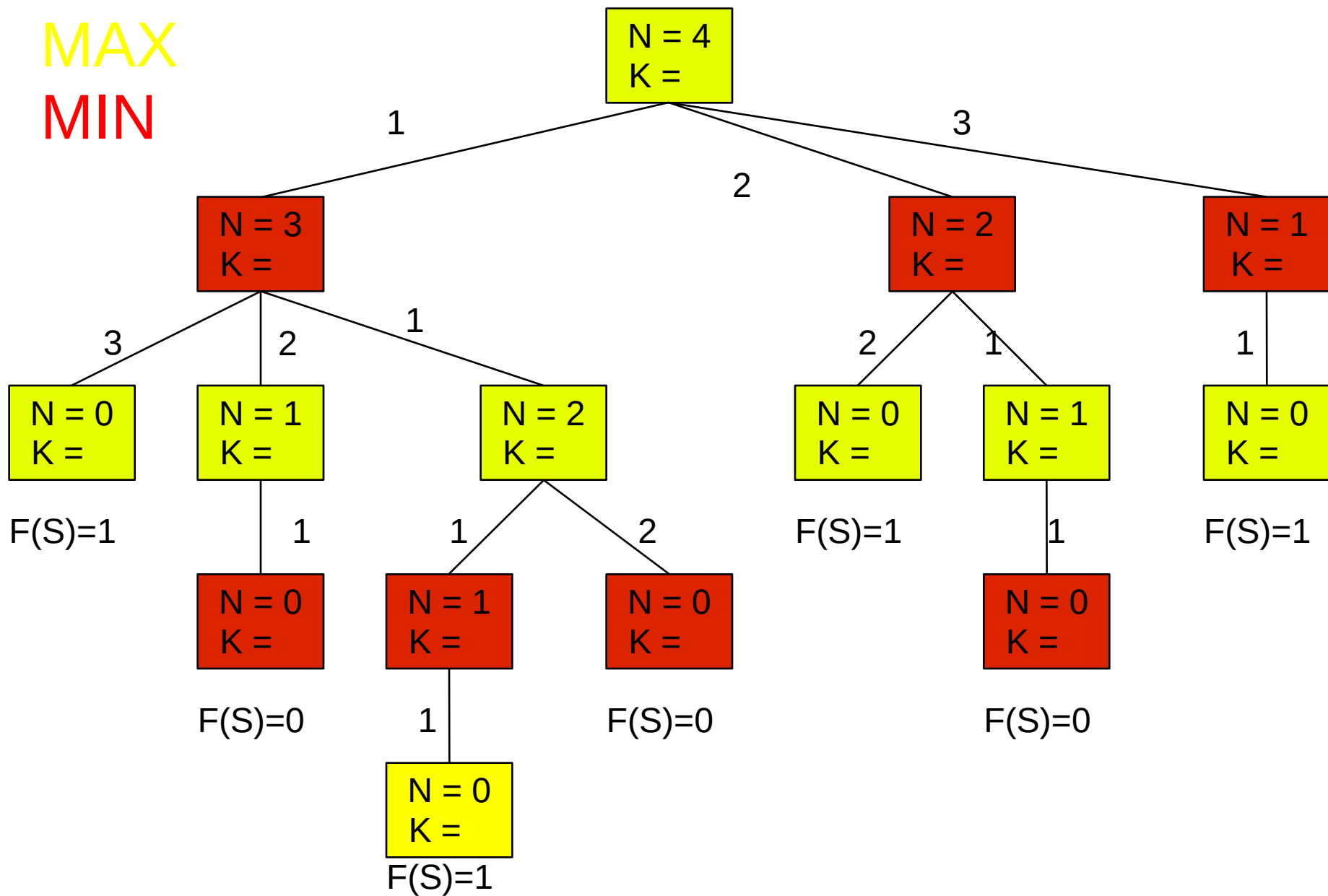
# Primer - Coins game

- Magacin sa  $N$  kovanica
- Kad je na potezu, igrač uzima 1, 2, ili 3 kovanice iz magacina
- Igrač koji uzme poslednju kovanicu gubi

# Coins Game: Formalna definicija

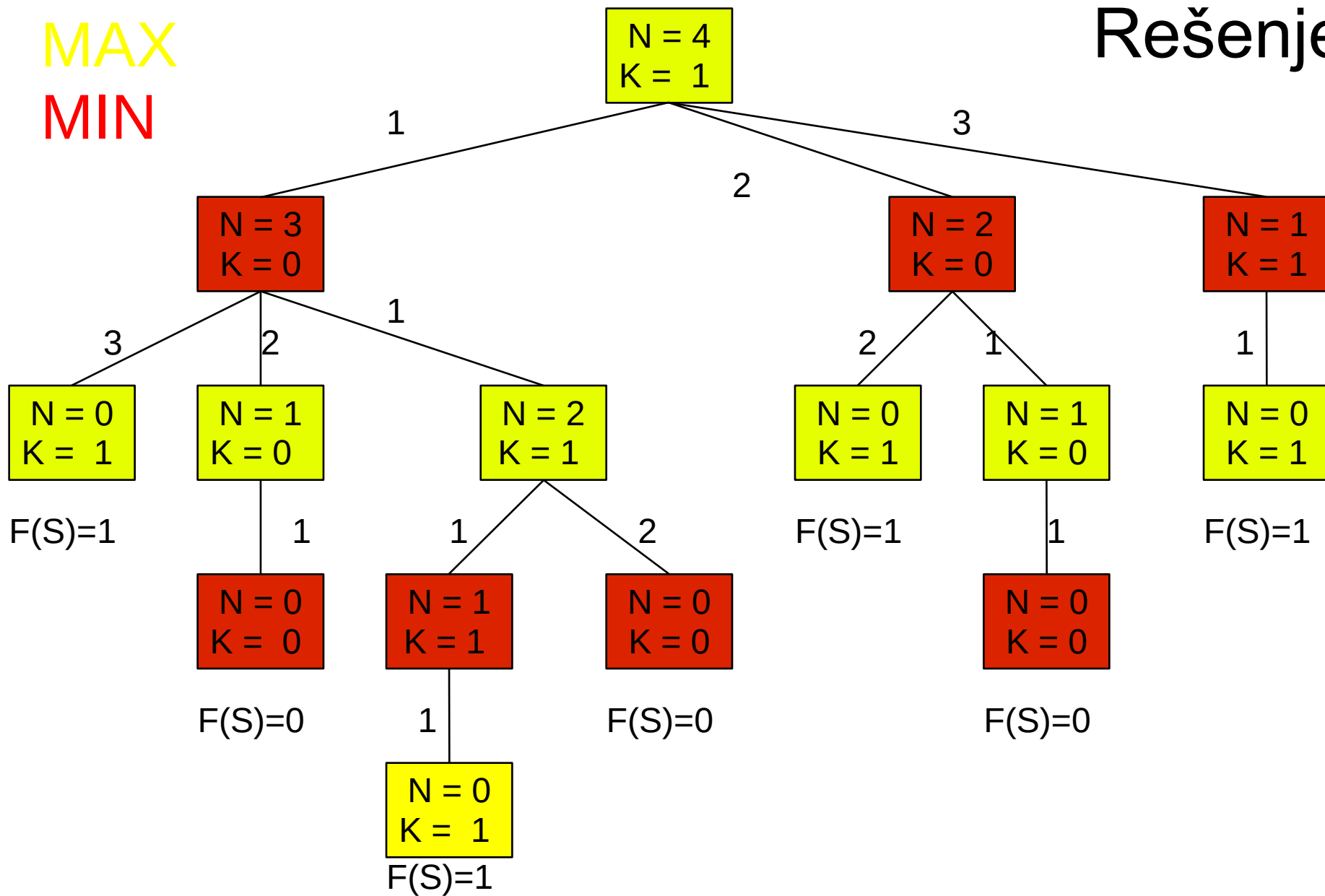
- Početno stanje: Broj kovanica u magacinu
- Operatori:
  1. Uzmi jednu kovanicu
  2. Uzmi dve kovanice
  3. Uzmi tri kovanice
- Test: Nema kovanica u magacinu
- Utility Function:  $F(S)$ 
  - ▣  $F(S) = 1$  ako MAX pobedi, 0 ako MIN pobedi

MAX  
MIN



MAX  
MIN

Rešenje



# Analiza

- Maks. Dubina ( $d$ ): 5
- Faktor grananja ( $b$ ): 3
- Broj čvorova: 15
- Iako je trivijalan primer, stablo je poprilično veliko
  - ▣ Generalno, ima  $O(b^d)$  čvorova za pretragu
  - ▣ Vreme izvršenja algoritma raste eksponencijalno!
  - ▣ Kako ga ubrzati?
- Ograničiti traženje po dubini
  - ▣ Pretražujete sve dok možete (vreme, zadata dubina)
  - ▣ Evaluacija stanja i vraćanje do korena

# MinMax ograničen po dubini

- Za čvor  $X$  koji se nalazi na dubini  $d$ :

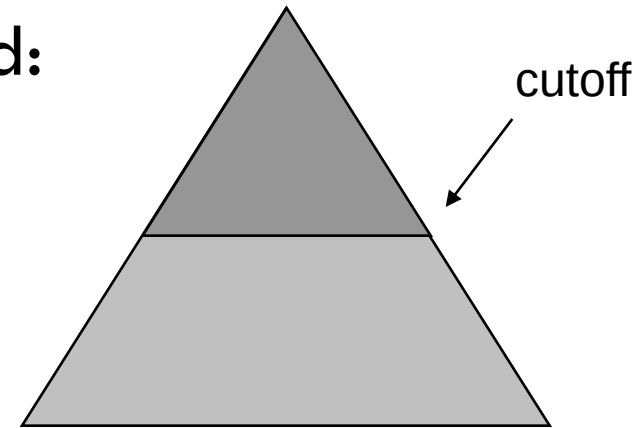
$$V(x, d) =$$

if ( $x$  je list) || ( $d > \text{max. dubina}$ )

- $P(x)$

else

- $\max \{V(c, d + 1) \mid c \text{ je dete čvora } x\}$  ako je potez tog igrača na redu.
- $\min \{V(c, d + 1) \mid c \text{ je dete čvora } x\}$  ako je potez drugog igrača na redu.



# IKS-OKS: Primer funkcije korisnosti

Ako Max igra kao "X"

$e(n) =$

if  $n$  je pobeda za Max,  $+\infty$

if  $n$  je pobeda za Min,  $-\infty$

else

(broj vrsta, kolona i dijagonala  
dostupnih za Max) - (broj vrsta,  
kolona i dijagonala dostupnih za  
Min)

|  |   |   |
|--|---|---|
|  |   |   |
|  | X | O |
|  |   |   |

$$e(n) = 6 - 4 = 2$$

|   |   |   |
|---|---|---|
|   |   |   |
| O | X | X |
|   | O |   |

$$e(n) = 4 - 3 = 1$$

# Šah: Primer funkcije korisnosti

Ako je Max: "White"

Pretpostavka da svaka figura ima sledeće vrednosti

pion = 1;

Kralj = 3;

Lovac = 3;

Top = 5;

Kraljica = 9;

$w$  = suma vrednosti belih figura

$b$  = suma vrednosti belih figura

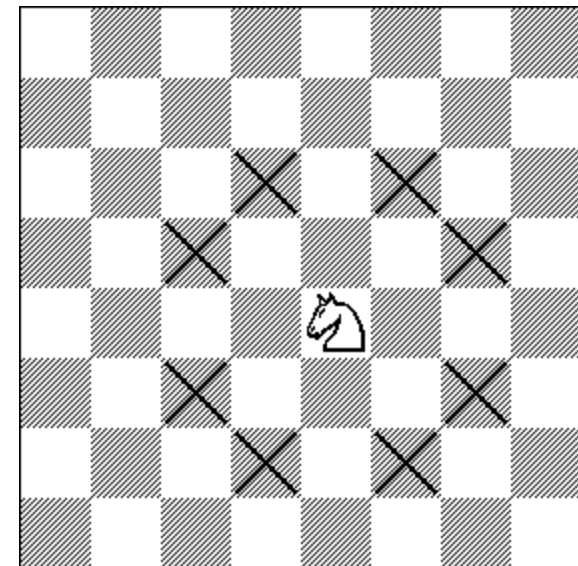
$$e(n) = \frac{w - b}{w + b}$$

Vrednost ide između 1 i -1

Ova funkcija naivno daje istu težinu figurama bez obzira na poziciju na tabli...

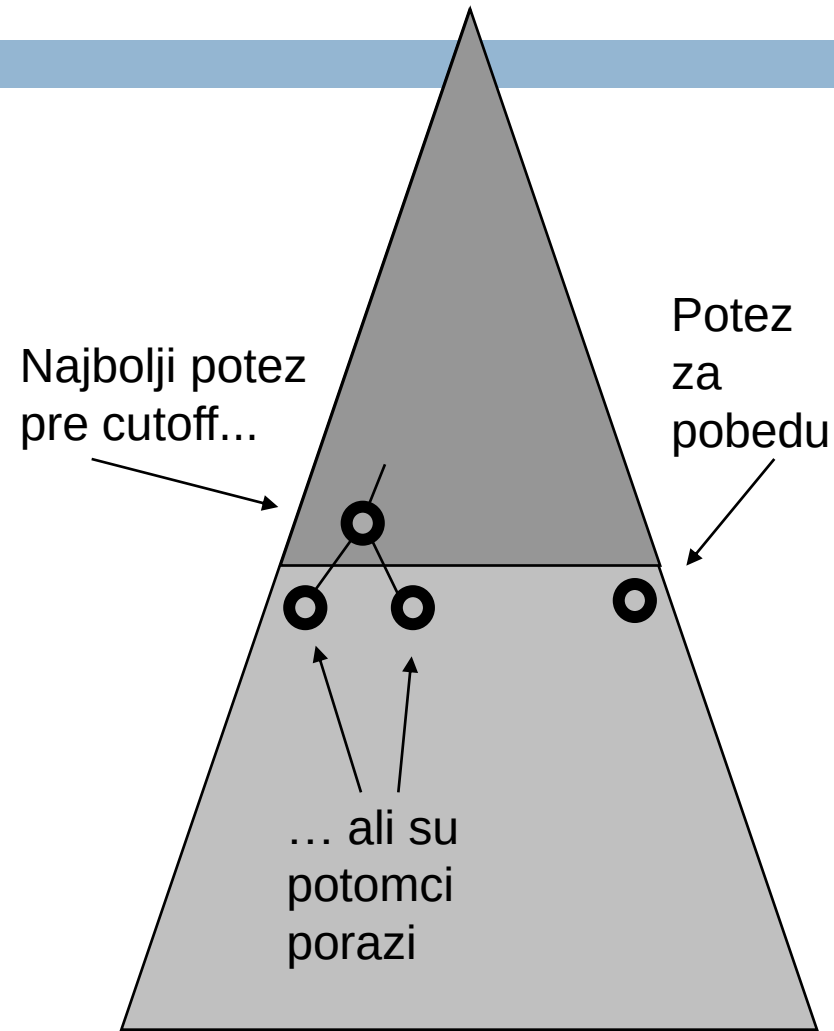
Kako doći do dobre funkcije?

- Intervjuisati eksperta
- Mašinsko učenje



# Alfa-beta odsecanje ( $\alpha$ - $\beta$ pruning)

- MinMax:
  - ▣ Zbog vremenskog ograničenja – ograničavanje po dubini
  - ▣ Korišćenje *cutoff* –a prouzrokuje probleme zbog tzv efekta “horizont”.
- Da li postoji način da se poveća dubina traženja za isto vreme?
- **Rešenje: Alpha-Beta Pruning...**



# Alfa-beta odsecanje

- Cilj algoritma: smanjiti stablo traženja
- Odseca grane koje ne obećavaju
- Osnovna ideja:
  - ▣ Izbegavaju se podstabla koja nemaju efekat na rezultat
  - ▣ Pamti se vrednost **najboljeg** poteza **do sada**
    - $\alpha$  – najbolja vrednost za igrača Max. Koristi se u MIN čvorovima, i dodeljuje u MAX čvorovima
    - $\beta$  – najbolja vrednost za igrača Min. Koristi se u MAX čvorovima i dodeljuje u MIN čvorovima
- Kada **Max** ispituje moguće akcije, ako je bilo koja od njih veća od  $\beta$  (sto je gore po Min), onda može da se prestane sa traženjem (podrazumeva se da Min neće odigrati potez koji nije dobar).

# Alpha-Beta odsecanje

## □ MAX (ne na nivou 0)

▣ Ako je nađeno podstablo sa vrednošću  $k$  koja je veća od vrednosti  $\beta$ , nema potrebe za njegovom daljom pretragom

■ MAX može da ima potez koji je dobar najmanje  $k$ , tako da MIN nikad neće da izabere taj put!

## □ MIN

▣ Ako je nađeno podstablo sa vrednošću  $k$  koja je manja od vrednosti  $\alpha$ , nema potrebe za njegovom daljom pretragom

■ MIN može da ima potez koji je dobar najmanje  $k$ , tako da MAX nikad neće da izabere taj put!

# Algoritam sa Alfa-beta odsecanjem

```
function ALPHA-BETA-DECISION(state) returns an action  
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))
```

---

```
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  inputs: state, current state in game  
          $\alpha$ , the value of the best alternative for MAX along the path to state  
          $\beta$ , the value of the best alternative for MIN along the path to state  
  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for a, s in SUCCESSORS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$   
    if  $v \geq \beta$  then return v  
     $\alpha \leftarrow \text{MAX}(\alpha, v)$   
  return v
```

---

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  same as MAX-VALUE but with roles of  $\alpha$ ,  $\beta$  reversed
```

# Algoritam sa Alfa-beta odsecanjem

Polazne vrednosti:  $\alpha = -\infty$ ,  $\beta = +\infty$

MAX-VALUE (state, game,  $\alpha$ ,  $\beta$ ) – vraca minimax za  
Max

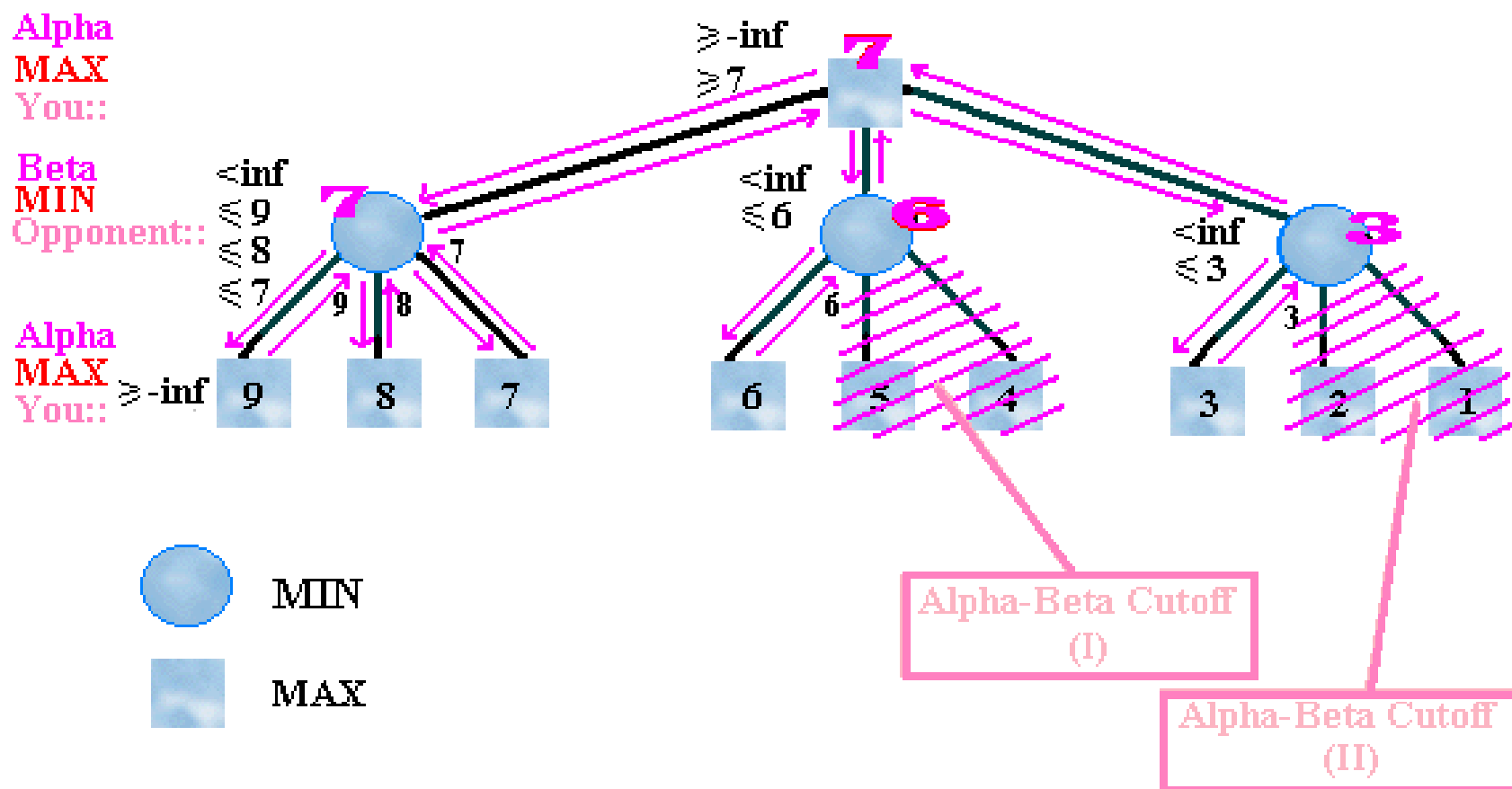
1. If cutoff-test(state) then return eval(state)
2. For each s in successors(state) do
3.      $\alpha := \max(\alpha, \text{min-value}(s, \text{game}, \alpha, \beta))$
4.     if  $\alpha \geq \beta$  then return  $\beta$
5. end
6. return  $\alpha$

# Algoritam sa Alfa-beta odsecanjem (2)

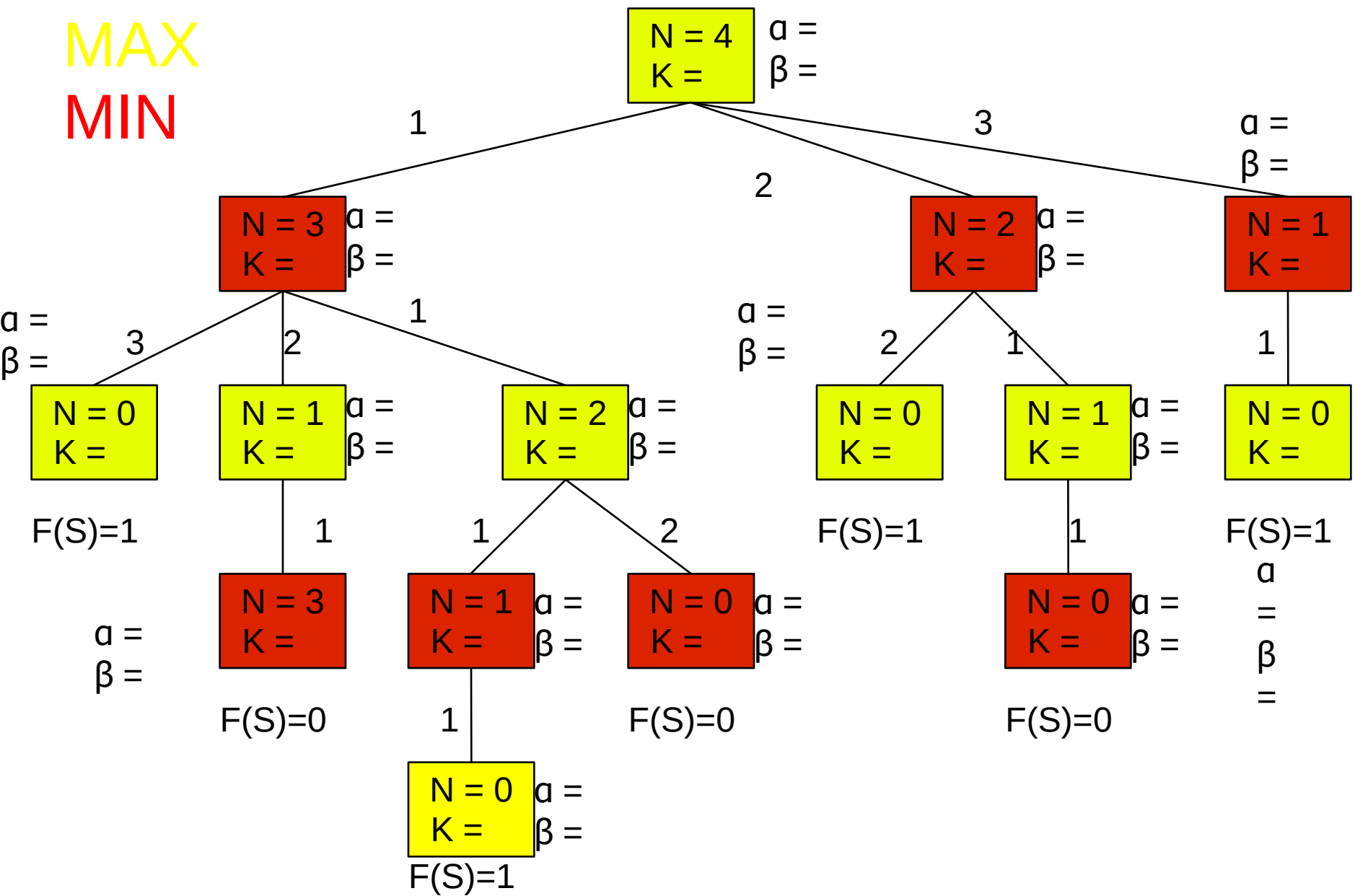
MIN-VALUE(state, game,  $\alpha$ ,  $\beta$ )

1. If cutoff-test(state) then return eval(state)
2. For each  $s$  in successors(state) do
3.      $\beta := \min(\beta, \max(\text{value}(s, \text{game}, \alpha, \beta))$
4.     If  $\beta \leq \alpha$  then return  $\alpha$
5. End
6. Return  $\beta$

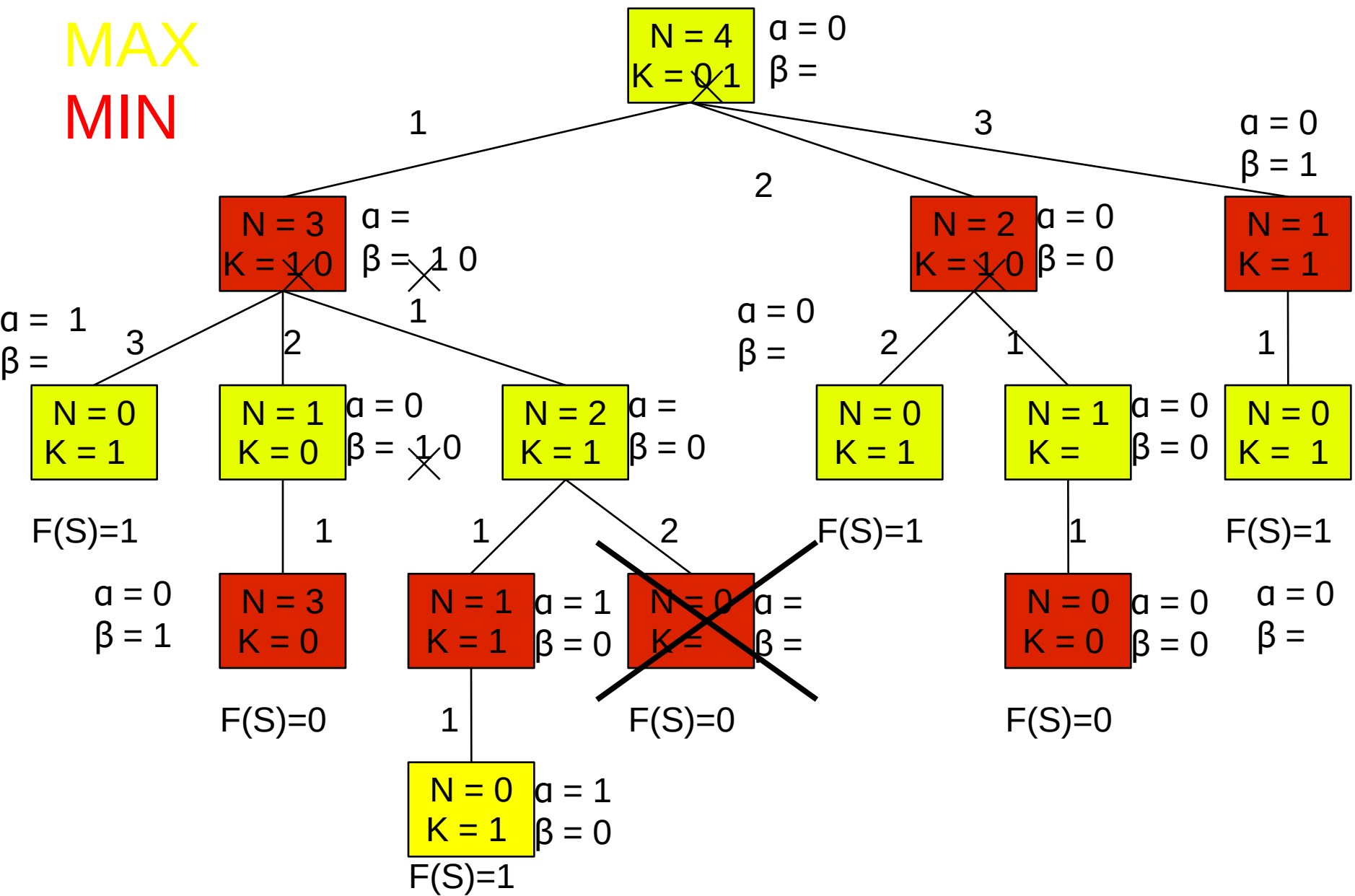
# Primer: AlfaBeta odsecanje



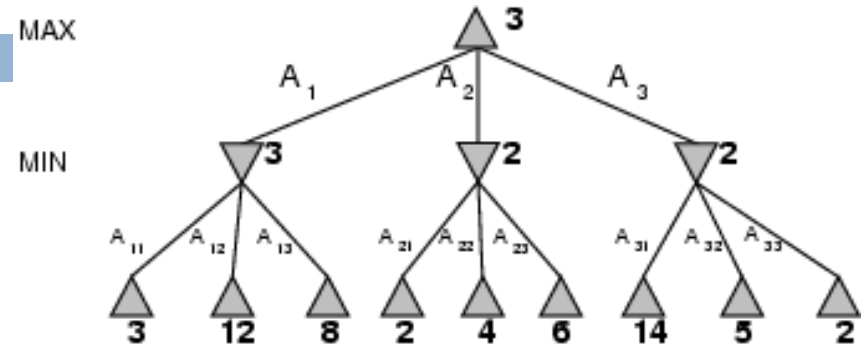
# MIN



MAX  
MIN

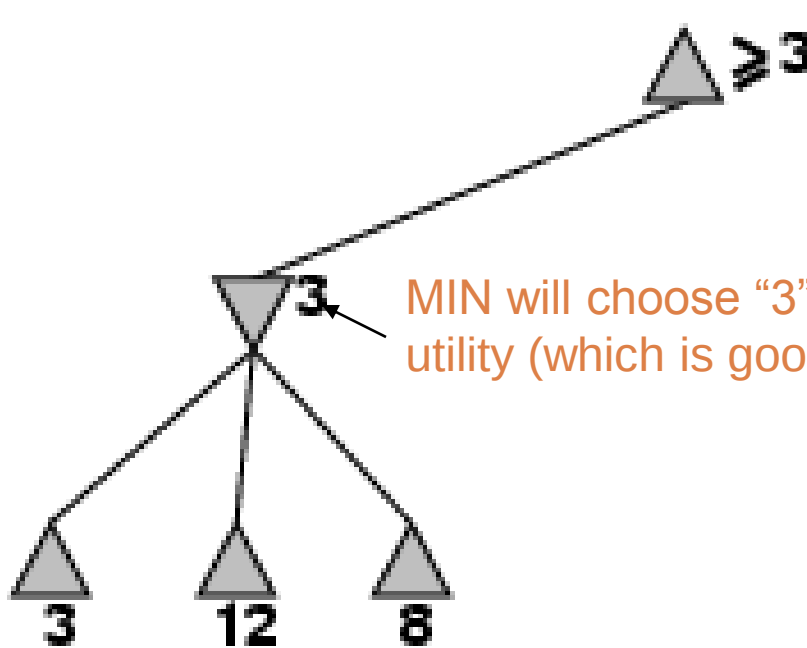


# Primer: $\alpha$ - $\beta$ pruning



MAX

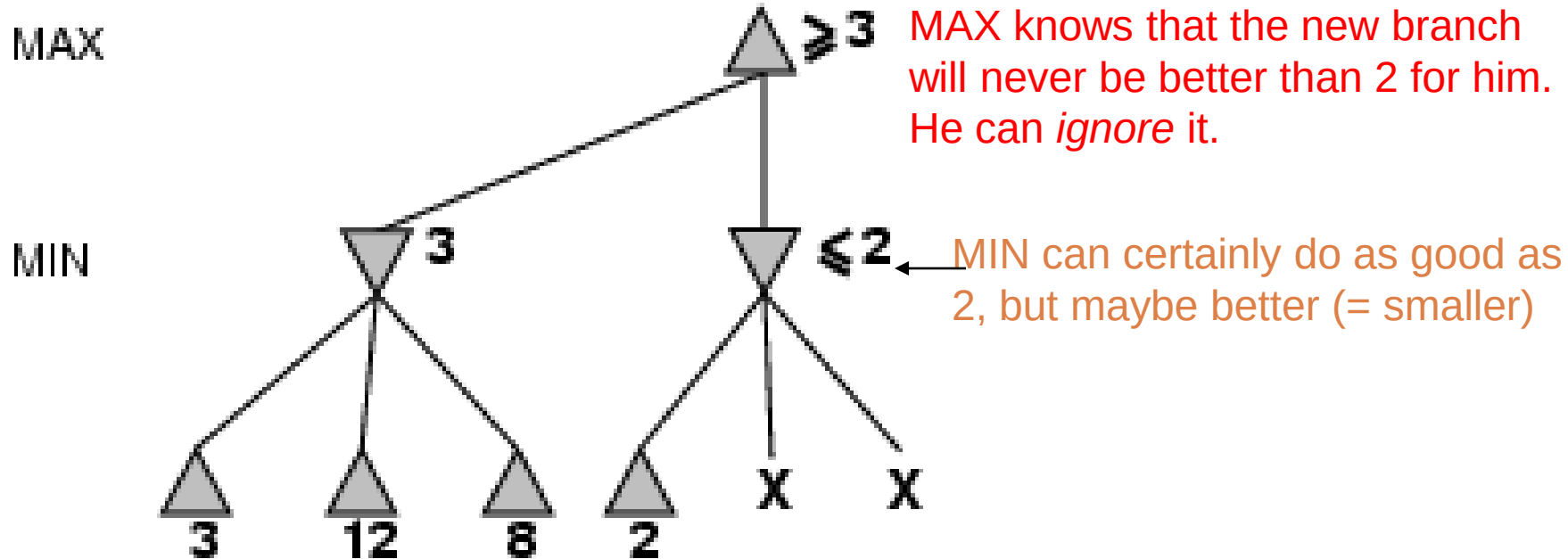
MIN



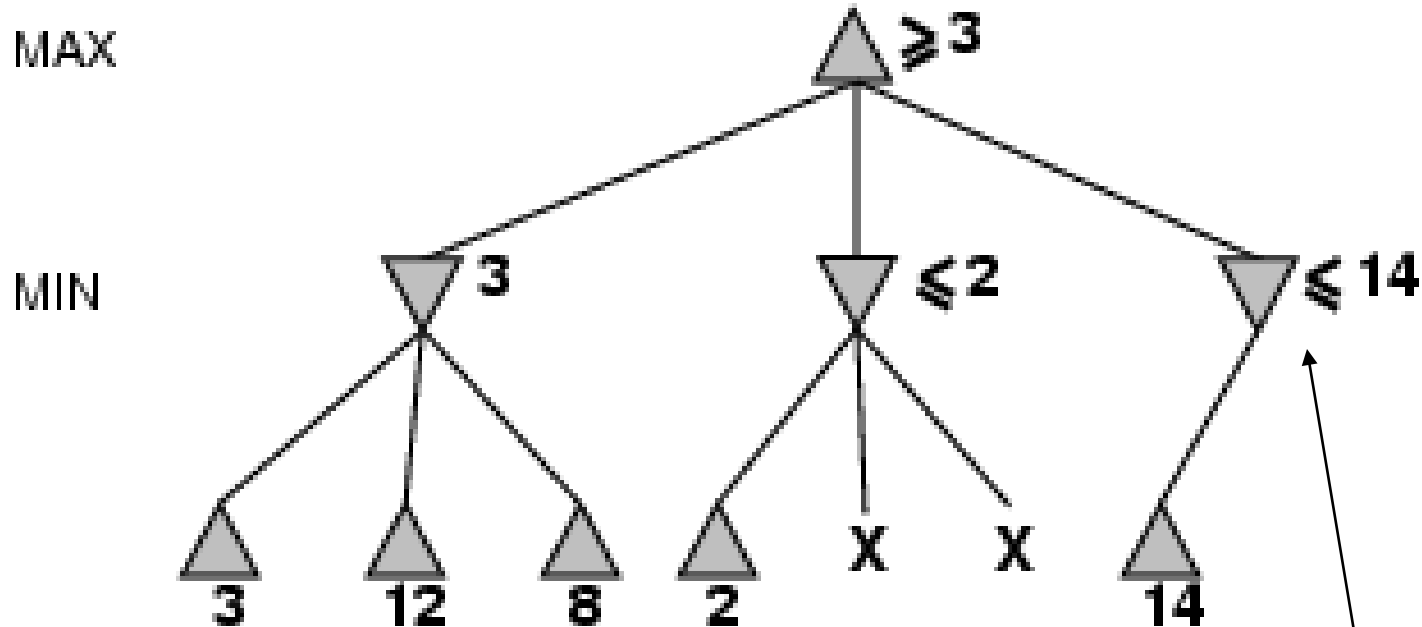
MAX knows that it can at least get "3" by playing this branch

MIN will choose "3", because it minimizes the utility (which is good for MIN)

# $\alpha$ - $\beta$ pruning example

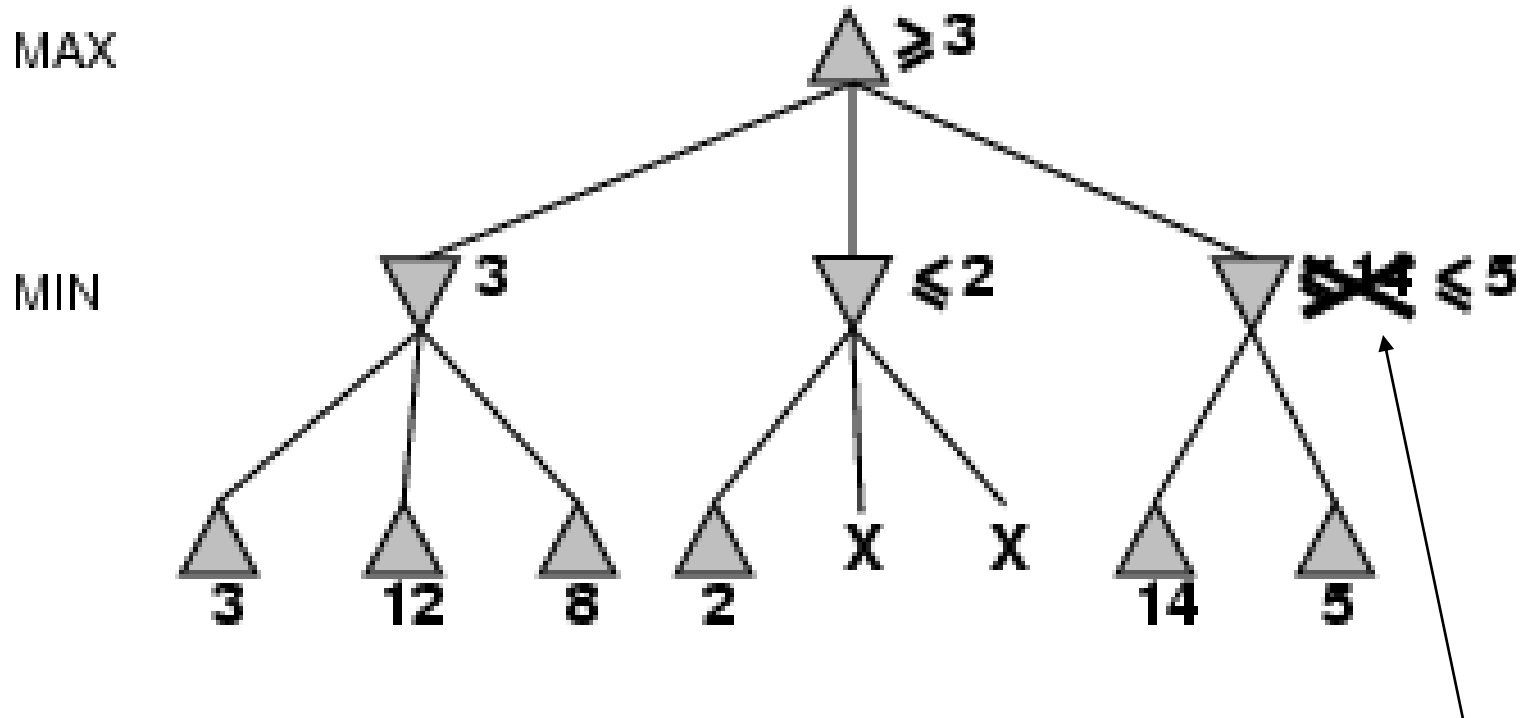


# $\alpha$ - $\beta$ pruning example



MIN will do at least as good as 14 in this branch (which is very good for MAX!) so MAX will want to explore this branch more.

# $\alpha$ - $\beta$ pruning example

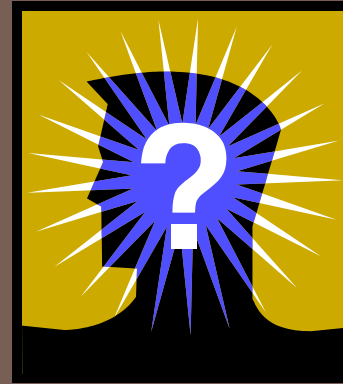


MIN will do at least as good as 5 in this branch (which is still good for MAX) so MAX will want to explore this branch more.

# Osobine $\alpha$ - $\beta$

- Odsecanje **ne** utiče na konačni rezultat.
- Dobar **redosled poteza** poboljšava efikasnost odsecanja  
(pogledati prethodni primer – poslednje odsecanje)
- Sa "perfektnim redosledom"  
vremenska kompleksnost =  $O(b^{m/2})$   
→ **duplira** dubinu traženja

# PITANJA?



## Dileme?

## Komentari?

